

Project 6 (C++) Scheduling. You are to implement the dependency graph, and scheduling. Your program will detect a cycle if the input graph contains cycles.

As taught in class, there are four options in scheduling:

- (1) using limited processors where each job takes 1 unit of processing time;
- (2) using limited processors where jobs take variable unit of processing time;
- (3) using unlimited processors where each job takes 1 unit of processing time;
- (4) using unlimited processors where jobs take variable unit of processing time.

This project is to implement an algorithm that handles the above four options into one program.

You will be given 3 test data sets, where each set includes two files: one contains the dependency graph and one contains processing time for each node. Nodes in the graphs represent jobs; in the specs, jobs and nodes means the same thing.

Set1: graph1 and jobTime1 // graph1 has 8 nodes, jobs take 1 unit of time.

Set2: graph2 and jobTime2 // graph2 has 10 nodes, jobs take variable times.

Set3: graph3 and jobTime3 // graph3 has 8 nodes, jobs take 1 unit of time; however, graph 3 contains a cycle.

What to do as follows:

- 1) Draw the dependency graph and schedule table with 3 processors for Set1.
- 2) Draw the dependency graph and schedule table with unlimited (10) processors for Set2.
- 3) Implement your program based on the specs given below.
- 4) Run and debug your program on Set1 with 3 processors until the results agree with your drawings in 1.
- 5) Run and debug your program on Set2 with unlimited processors until the results agree with your drawing in 2.
- 6) Run and debug your program on Set3 with unlimited processors until the cycle is detected.

*** Include in your hard copies:

- cover page
- your drawing for Set1 (-2 if omitted.)
- your drawing for Set2 (-2 if omitted.)
- source code
- graph1 // with caption: "**** Below is graph1."
- jobTime1 // with caption: "**** Below is the processing times for nodes in graph1."
- outFile1 from graph1 and jobTime1 // with caption: "**** Below is outFile1 for graph1."
- logFile from graph1 and jobTime1 // with caption: "**** Below is logFile for graph1"; limit to 3 pages if more.
- graph2 // with caption: "**** Below is graph2"
- jobTime2 // with caption: "**** Below is the processing times for nodes in graph2."
- outFile1 from graph2 and jobTime2 // with caption: "**** Below is outFile1 for graph2."
- logFile from graph2 and jobTime2 // with caption: "**** Below is logFile for graph2"; limit to 3 pages if more.
- graph3 // with caption: "**** Below is graph3"
- jobTime3 // with caption: "**** Below is the processing times for nodes in graph3."
- outFile1 from graph3 and jobTime3 // with caption: "**** Below is outFile1 for graph3."
- logFile from graph3 and jobTime3 // with caption: "**** Below is logFile for graph3"; limit to 3 pages if more.

Language: C++

Project name: Dependency graph and Scheduling

Points: 12 points

Due Date: Soft copy (*.zip) and hard copy (word file):

(12/12 pts): on time, 4/5/2026 Sunday before midnight, 11:59pm.

(-12/12 pts): non-submission, 4/5/2026 Sunday after midnight

*** All submission MUST include soft copy (*.zip) and hard copy (*.docx) in the same email attachments with correct email subject as stated in the project submission requirement; otherwise, your submission will be rejected.

Email subject: (323) firstName lastName <Project 6: Dependency graph and Scheduling (C++)>

*** Inside the email body includes:

- Your answer to the five questions (-2 if omitted)
- Screen recoding link. (-2 without the recording!)

*** Place your screen recording link in your project submission email body below the 5 questions.

I. Inputs: There are three (3) inputs to the program.

a) inFile1 (use argv [1]): a text file representing the dependency graph, $G = \langle N, E \rangle$.

The first number in inFile1 is the number of nodes in the graph; then follows by a list of directed edges (dependency) $\langle n_i, n_j \rangle$, where n_i, n_j are node's IDs; node's IDs are from 1 to Num Nodes, 0 is not used.

For example of an input graph:

```
6          // Graph has 6 nodes
4 3        // a dependency edge from 4 to 3 (i.e., 3 is a dependent of 4 & 4 is a parent of 3.)
4 5        // a dependency edge from 4 to 5 (i.e., 5 is a dependent of 4 & 4 is a parent of 5.)
6 1        // ...
1 2        // ...
:
```

b) inFile2 (use argv [2]): a text file contains the processing times for nodes.

The first number in inFile2 is the number of nodes in the graph;

then follows by a list of pairs, $\langle n_i, t_i \rangle$, where n_i is the node's id and t_i is the unit of processing times for node n_i .

For example: all jobs take the same unit of processing time;

```
6          // Graph has 6 nodes
1 1        // job time for node 1 is 1
2 1        // job time for node 2 is 1
3 1        // job time for node 3 is 1
:
```

another example: jobs take variable of processing time

```
6          // there are 6 nodes in the input graph
1 3        // job time for node 1 is 3
2 4        // job time for node 2 is 4
3 1        // job time for node 3 is 1
:
```

c) number processors (use argv [3]) // according to each run as described in the above.

II. Outputs: There are two (2) output files

a) outFile1: (use argv [4]) for the intermediate and final results of the schedule table, nicely formatted.

For example, if there are 7 nodes in the graph and currentTime is 4 and 2 processors are used so far, then print to outFile1 as below.

```
=====
ProcUsed : 2   currentTime: 4

Time:      0  1  2  3  4  5  6  7
-----
Proc: 1   | 1 | 1 | 7 | 5 | 5 |   |   |
-----
Proc: 2   | 2 | 4 | 4 | 4 | - |   |   |
-----

Open (Time 4): (id, time) → (id, time) → (id, time) → ...
=====
```

b) logFile (use argv [5]): as program dictates.

III. Data structure:

- A listNode class

- (int) jobID
 - (int) jobTime
 - (listNode*) next
- Method:
- constructor (...)

- A schedule class

- (int) numNodes // the total number of nodes in the input graph.
- (int) numProcs // the upper limit of processors can be used.
- (int) procUsed // number of processors used so far; **initialized to 0.**
- (int) currentTime // **initialize to 0.**
- (int) totalJobTimes // the sum of all job times of nodes in the graph.
- (int *) jobTimeAry // an 1D array to store the job time of each node in the graph;
// to be dynamically allocated, size of numNodes +1; initialized to 1.
- (int *) jobStatus // an 1D array to store the status of each node in the graph;
// jobStatus[i] == 0 means node i is not in the graph. (i.e., done and deleted.)
// jobStatus[i] == 1 means node i is in the graph and not marked yet.
// jobStatus[i] == 2 means node i is marked, but not done yet.
// to be dynamically allocated, size of numNodes +1; initialized to 1.
- (int **) Matrix // a 2-D integer array, size numNodes+1 by numNodes+1,
// to represent the dependency graph; **to be dynamically allocated, initialized to zero.**
// The usage of this matrix as follows:
// Matrix [0][0] to store number of nodes remain in the graph; initialize to numNodes;
// decreases by 1, when a node is deleted; So, to check if the graph is empty,
// just check if Matrix [0][0] == 0.
// Matrix [i][j] >= 1, means node i is a parent of node j (i.e., node j is a dependent of node i).

- // Matrix [0][j], we use row 0 to store the parent counts of node j.
- //The parent counts of node j is the total count of none zero rows in **j-th column**.
- // So if we want to know the parent counts of node j, we check Matrix[0][j].
- // if Matrix[0][j] == 0 means node j is an orphan.
- // Matrix [i][0], we use column 0 to store the dependent counts of node i.
- //The dependent counts of node i is the total count of none zero columns in **i-th row**.
- // So if we want to know the dependent counts of node i, we check Matrix [i][0].
- (int **) Table // a 2-D integer array, size of (numProcs +1) by (totalJobTimes +1) for scheduling;
- // **to be dynamically allocated, and initialized to 0.**
- // Index of Table row represents processor's id, and index of Table column represents time slots,
- // and, the content of Table [i][T] tells whether processor is working on a job or not, i.e.,
- // Table [i][T] > 0 means the processor i, at time slot T, is processing job, Table[i][T].
- // Table [i][T] <= 0 means the processor i is idled (available) at time slot T.
- (listNode*) Open // Open acts as the list head of a linked list points to a dummy node.
- // Nodes in Open are sorted in ascending order by jobID.
- // **Initialized points to a dummy listNode (-1, 0).** //Dummy node's ID is -1 and jobTime is 0.

Methods:

- constructor (...) // does allocations and initializations of all schedule class members (**see the list in the above.**)
- loadMatrix (inFile1) // read an edge <n_i, n_j> from inFile1 and set Matrix [n_i, n_j] ← 1
- setMatrix (...) // **See algorithm below.** // If you like, let the construct call this method.
- countParent (...) // **See algorithm below.**
- countDependent (...) // **See algorithm below.**
- (int) loadJobTimeAry (inFile2) // read each pair <jobID, time> from inFile2 and load to jobTimeAry;
- // set jobTimeAry[jobID] ← time; this method needs to keep track of total job times;
- // it returns totalJobTimes
- (int) findOrphan (...)
 - // Check Matrix [0][j], the 0-th row, to find the next un-marked orphan node, j, i.e., Matrix [0][j] == 0
 - //&& jobStatus[j] == 1. If found, mark the orphan, i.e., set jobStatus[j] ← 2, then returns j;
 - // if no such j, returns -1.
- (listNode*) findspot (...) //The method is similar to the method in your previous projects, the nodes in the linked
- //list are sorted in ascending order with respected to node's ID. The method locates the spot where
- //newNode will be inserted in the list and it returns spot. The method does not do insertion!
- (int) getNextProc (currentTime) // on your own
 - // check Table [i][currentTime], i from 0 to (numProc -1) to find the first i processor where Table
 - // [i][currentTime] == 0
 - // if found returns i, else returns -1, means no available processor.
- fillOPEN (...) // populate OPEN from orphan nodes in the graph; **see algorithm below.**
- fillTable (...) // populate the table from jobs in OPEN; **see algorithm below.**
- putJobOnTable (...) // **see algorithm below.**
- printMatrix (...) // Print the entire content of Matrix with row and column indices in readable format.
- printOPEN (...) // Prints to file, nodes in OPEN linked list; for each node, print node's ID, node's JobTime, and
- // the next node's ID, in the following format:
- // OPEN → (-1, 0, next's id) → (id, time, next's id) →
- // OPEN → (-1,0,2) → (2,...) → ...
- (listNode *) removeOPEN () //Remove the front node of OPEN after dummy node if OPEN is not empty
- // if OPEN's next is not null
 - tmp ← OPEN's next
 - OPEN's next ← tmp's next
 - tmp's next ← null
 - return tmp
- else
- return null

```

- printTable (outFile1, currentTime) // print the Table up to the currentTime slot to outFile1,
    // On your own, see the format description given in the above.
- print1DAry (Ary, oFile) // print i and Ary[i] to oFile, one pair per one text-line, i from 0 to numNodes. i.e.,
    0      whatever in Ary[0]
    1      whatever in Ary[1]
    2      whatever in Ary[2]
    :      :
    :      :
    numNodes  whatever in Ary[numNodes]

- (bool) checkCycle (...)
    Check the followings: if all 3 conditions below are true, returns true, else returns false.
    (1) OPEN is empty.
    (2) Graph is not empty. // Matrix [0][0] > 0
    (3) all processors are available.
    // check Table [i][currentTime], i from 0 to (numProc -1) where all Table [i][currentTime] == 0

- (bool) isGraphEmpty (...) // if Matrix [0][0] == 0 returns true, else returns false.
    //When you printMatrix, pay attention to the content of Matrix [0][0].

- deleteDoneJobs (...) // finds all done jobs on Table then call deleteJob () to delete; see algorithm steps below.

- deleteJob (i) // see algorithm steps below.
    // When a job is done, we delete the job i and its outgoing edges from the graph, as follows.
    // 1) To delete a job i from the graph is to set jobStatus[i] to 0 and decrease the
    //    number of nodes in graph by 1, i.e., Matrix [0][0] --
    // 2) To delete a job's outgoing edges is to decrease the parent counts of all i's dependents by 1.
    //    Those job i's dependents are those Matrix [i][j] > 0, j is a dependent of i; i is a parent of j.
    //    So, for all Matrix[i][j] > 0, we decrease Matrix [0][j] by 1.

```

IV. main(..)

```

/**
/** First, only implement Step 1 and Step 2; check outFile1 to see if all outputs are correct, only if correct, then proceed
/** to implement Step 3 and Step 4; check outFile1 to see if all outputs of OPEN and Table are correct, only if all correct,
/** then proceed to implement all Steps.
/**

```

Step 1: check correctness of argc count and each file can be opened.

```

inFile1, inFile2, outFile1, logFile ← open via argv []
numNodes ← read from inFile1
numProcs ← get from argv []
if (numProcs <= 0) exit with error message “need 1 or more processors”.
    else if (numProcs > numNodes)
        numProcs ← numNodes // means unlimited processors.

```

Constructor does all allocations and initialization of all members in schedule class:

```

jobTimeAry ← dynamically allocated, size of numNodes +1; initialized to 1.
jobStatus ← dynamically allocated, size of numNodes +1; initialized to 1.
Matrix ← dynamically allocated, size of numNodes+1 by numNodes+1, initialized to zero.
Table ← dynamically allocated, size of numNodes+1 by numNodes+1, initialized to zero.
OPEN ← use new to get a dummy listNode (-1, 0) for it to point to.

```

Step 2: $\text{currentTime} \leftarrow 0$ // at the beginning of scheduling
 $\text{procUsed} \leftarrow 0$ // no processor is used at the beginning
 $\text{loadMatrix}(\text{inFile1})$
 $\text{setMatrix}(\dots)$
 $\text{outFile1} \leftarrow \text{"*** in main (), printing Matrix"}$
 $\text{printMatrix}(\text{outFile1})$ // check for yourself to make sure the matrix is correctly loaded.
 $\text{totalJobTimes} \leftarrow \text{loadJobTimeAry}(\text{inFile2})$
 $\text{outFile1} \leftarrow \text{"*** in main (), printing jobTimeAry"}$
 $\text{print1DAry}(\text{jobTimeAry}, \text{outFile1})$
 $\text{outFile1} \leftarrow \text{"*** in main (), printing jobStatus"}$
 $\text{print1DAry}(\text{jobStatus}, \text{outFile1})$

Step 3: $\text{outFile1} \leftarrow \text{"In main (), printing OPEN, before fillOPEN"}$
 $\text{printOPEN}(\text{outFile1})$
 $\text{fillOPEN}(\dots)$
 $\text{outFile1} \leftarrow \text{"In main (), printing Table, after fillOPEN"}$
 $\text{printOPEN}(\text{outFile1})$

Step 4: $\text{outFile1} \leftarrow \text{"In main (), printing Table, before fillTable"}$
 $\text{printTable}(\text{outFile1}, \text{currentTime})$
 $\text{fillTable}(\dots)$
 $\text{outFile1} \leftarrow \text{"In main (), printing Table, after fillTable"}$
 $\text{printTable}(\text{outFile1}, \text{currentTime})$

Step 5: $\text{currentTime}++$

Step 6: $\text{deleteDoneJobs}(\dots)$

Step 7: $\text{cycle} \leftarrow \text{checkCycle}(\dots)$ if cycle is true
 $\text{outFile1} \leftarrow \text{"there is cycle in the graph!!!"}$;
close all files and exit the program

Step 8: repeat step 3 to step 6 until $\text{isGraphEmpty}()$

Step 9: $\text{outFile1} \leftarrow \text{"In main () printing the final table"}$
 $\text{printTable}(\text{outFile1})$ // The final schedule table.

step 10: close all files

V. $\text{setMatrix}(\text{Matrix}, \text{logFile})$

Step 0: $\text{logFile} \leftarrow \text{"Entering setMatrix method"}$

Step 1: $\text{Matrix}[0][0] \leftarrow \text{numNodes}$

Step 2: $j \leftarrow 1$

Step 3: $\text{Matrix}[0][j] \leftarrow \text{countParent}(\text{Matrix}, j, \text{logFile})$
 $\text{Matrix}[j][0] \leftarrow \text{countDependent}(\text{Matrix}, j, \text{logFile})$

Step 4: $j++$

Step 5: repeat step 3 to step 4 until $j > \text{numNodes}$

Step 6: $\text{logFile} \leftarrow \text{"Leaving setMatrix ()"}$

VI. $\text{countParent}(\text{Matrix}, j, \text{logFile})$

Step 0: $\text{logFile} \leftarrow \text{"Entering countParent () method"}$

Step 1: $\text{count} \leftarrow 0$

$i \leftarrow 1$

Step 2: if $\text{Matrix}[i][j] > 0$
 $\text{count}++$

Step 3: $i++$

Step 4: repeat step 2 to step 3 while $i \leq \text{numNodes}$

Step 5: $\text{logFile} \leftarrow \text{"In countParent () parent count is"}$ // print count

Step 6: return count

VII. countDependent (Matrix, j, logFile)

Step 0: logFile $\leftarrow$ "Entering countDependent () method"

Step 1: count $\leftarrow$ 0

 i $\leftarrow$ 1

Step 2: if Matrix[j][i] > 0

 count++

Step 3: i++

Step 4: repeat step 2 to step 3 while i <= numNodes

Step 5: logFile $\leftarrow$ "In countDependent () dependent count is" // print count

Step 6: return count

VIII. fillOPEN (logFile)

Step 0: logFile $\leftarrow$ "Entering fillOPEN () method"

Step 1: jobID $\leftarrow$ findOrphan (...)

Step 2: if jobID > 0

 newNode $\leftarrow$ get a node with (jobID, jobTimeAry[jobID])

 Spot $\leftarrow$ findspot (OPEN, newNode)

 newNode's next $\leftarrow$ Spot's next

 Spot's next $\leftarrow$ newNode

 logFile $\leftarrow$ "in fillOPEN (), printing OPEN"

 printOPEN(logFile)

Step 3: repeat step 1 to step 2 until no more unmarked orphan

Step 4: logFile $\leftarrow$ "Leaving fillOPEN () method"

IX. fillTable (logFile)

Step 0: logFile $\leftarrow$ "Entering fillTable () method"

Step 1: availProc $\leftarrow$ getNextProc (currentTime)

 if availProc >= 0 && OPEN's next != null // means there is a processor available and OPEN is not empty

 {newJob $\leftarrow$ removeOPEN ()

 putJobOnTable (availProc, currentTime, newJob's jobID, newJob's jobTime)

 if availProc >= procUsed

 procUsed ++

 }

Step 2: logFile $\leftarrow$ "In fillTable (), printing the Table; availProc = ; procUsed =" //Write those two values.

 printTable (logFile, currentTime)

Step 3 repeat step 1 and step 2 while availProc >= 0 && OPEN is not empty && procUsed < numProcs

Step 4: logFile $\leftarrow$ "Leaving fillTable () method"

X. putJobOnTable (availProc, currentTime, jobId, jobTime, logFile)

Step 0: logFile $\leftarrow$ "Entering putJobOnTable () method"

Step 1: Time $\leftarrow$ currentTime

 EndTime $\leftarrow$ Time + jobTime

Step 2: Table[availProc][Time] $\leftarrow$ jobId

Step 3: Time ++

Step 4: repeat step 2 to step 3 while Time < EndTime

Step 5: logFile $\leftarrow$ "Leaving putJobOnTable () method"

XI. deleteDoneJobs (logFile)

Step 0: logFile $\leftarrow$ “Entering deleteDoneJobs () method”

Step 1: proc $\leftarrow$ 0

Step 2: if Table[proc][currentTime] $\leq$ 0 && Table[proc][currentTime - 1] $>$ 0
 // the processor, proc, just finished a job in the previous time cycle.
 {jobID $\leftarrow$ Table[proc][currentTime - 1]
 deleteJob (jobID, logFile)
 }

Step 3: proc ++

Step 4: repeat step 2 to step 4 while proc $<$ numProcs

Step 5: logFile $\leftarrow$ “Leaving deleteDoneJobs () method”

XII. deleteJob (jobID, logFile) // When a job is done, we delete the job and its outgoing edges.

Step 0: logFile $\leftarrow$ “Entering deleteJob () method”

Step 1: jobStatus [JobID] $\leftarrow$ 0 // delete node (jobID) from the graph
 Matrix [0][0] -- // one less node in the graph

Step 2: j $\leftarrow$ 1

Step 3: if Matrix [jobID][j] $>$ 0 // if j is a dependent of jobID
 Matrix [0][j] -- // delete node's outgoing edges by decrease j's parent count by 1

Step 4: j ++

Step 5: repeat Step 3 to Step 4 while j $\leq$ numNodes

Step 6: logFile $\leftarrow$ “Leaving deleteJob () method”